

A-NESI: A Scalable Approximate Method for Probabilistic Neurosymbolic Inference

Emile van Krieken
University of Edinburgh
Vrije Universiteit Amsterdam
Emile.van.Krieken@ed.ac.uk

Thiviyan Thanapalasingam
University of Amsterdam

Jakub M. Tomczak
Eindhoven University of Technology

Frank van Harmelen, Annette ten Teije
Vrije Universiteit Amsterdam

Abstract

We study the problem of combining neural networks with symbolic reasoning. Recently introduced frameworks for Probabilistic Neurosymbolic Learning (PNL), such as DeepProbLog, perform exponential-time exact inference, limiting the scalability of PNL solutions. We introduce *Approximate Neurosymbolic Inference* (A-NESI): a new framework for PNL that uses neural networks for scalable approximate inference. A-NESI 1) performs approximate inference in polynomial time without changing the semantics of probabilistic logics; 2) is trained using data generated by the background knowledge; 3) can generate symbolic explanations of predictions; and 4) can guarantee the satisfaction of logical constraints at test time, which is vital in safety-critical applications. Our experiments show that A-NESI is the first end-to-end method to solve three neurosymbolic tasks with exponential combinatorial scaling. Finally, our experiments show that A-NESI achieves explainability and safety without a penalty in performance.

1 Introduction

Recent work in neurosymbolic learning combines neural perception with symbolic reasoning [52, 38], using symbolic knowledge to constrain the neural network [19], to learn perception from weak supervision signals [36], and to improve data efficiency [5, 57]. Many neurosymbolic methods use a differentiable logic such as fuzzy logics [5, 17] or probabilistic logics [35, 57, 16]. We call the latter *Probabilistic Neurosymbolic Learning* (PNL) methods. PNL methods add probabilities over discrete truth values to maintain all logical equivalences from classical logic, unlike fuzzy logics [54]. However, performing inference requires solving the *weighted model counting* (WMC) problem, which is computationally exponential [13], significantly limiting the kind of tasks that PNL can solve.

We study how to scale PNL to exponentially complex tasks using deep generative modelling [51]. Our method called *Approximate Neurosymbolic Inference* (A-NESI), introduces two neural networks that perform approximate inference over the WMC problem. The *prediction model* predicts the output of the system, while the *explanation model* computes which worlds (i.e. which truth assignment to a set of logical symbols) best explain a prediction. We use a novel training algorithm to fit both models with data generated using background knowledge: A-NESI samples symbol probabilities from a prior and uses the symbolic background knowledge to compute likely outputs given these probabilities. We train both models on these samples. See Figure 1 for an overview.

A-NESI combines all benefits of neurosymbolic learning with scalability. Our experiments on the Multi-digit MNISTAdd problem [35] show that, unlike other approaches, A-NESI scales almost

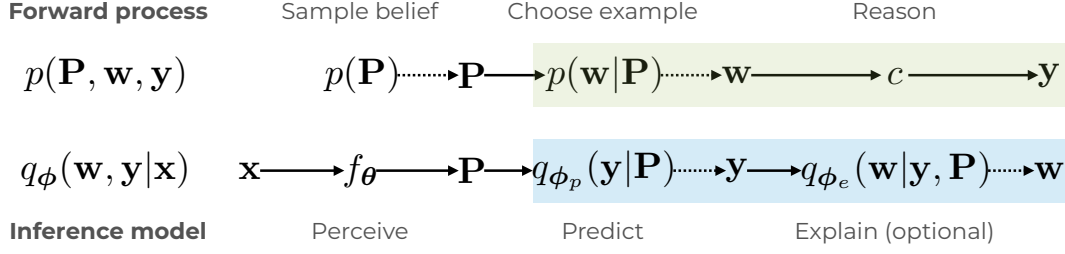


Figure 1: Overview of A-NESI. The forward process samples a belief $\mathbf{P}$ from a prior, then chooses a world $\mathbf{w}$ for that belief. The symbolic function c computes its output $\mathbf{y}$. The inference model uses the perception model f_θ to find a belief $\mathbf{P}$, then uses the prediction model q_ϕ to find the most likely output $\mathbf{y}$ for that belief. If we also use the optional explanation model, then q_ψ explains the output.

linearly in the number of digits and solves MNISTAdd problems with up to 15 digits while maintaining the predictive performance of exact inference. Furthermore, it can produce explanations of predictions and guarantee the satisfaction of logical constraints using a novel symbolic pruner.

The paper is organized as follows. In Section 3, we introduce A-NESI. Section 3.1 presents scalable neural networks for approximate inference. Section 3.2 outlines a novel training algorithm using data generated by the background knowledge. Section 3.2.4 extends A-NESI to include an explanation model. Section 3.3 extends A-NESI to guarantee the satisfaction of logical formulas. In Section 4, we perform experiments on three Neurosymbolic tasks that require perception and reasoning. Our experiments on Multi-digit MNISTAdd show that A-NESI learns to predict sums of two numbers with 15 handwritten digits, up from 4 in competing systems. Similarly, A-NESI can classify Sudoku puzzles in 9×9 grids, up from 4, and find the shortest path in 30×30 grids, up from 12.

2 Problem setup

First, we introduce our inference problem. We will use the MNISTAdd task from [35] as a running example. In this problem, we must learn to recognize the sum of two MNIST digits using only the sum as a training label. Importantly, we do not provide the labels of the individual MNIST digits.

2.1 Problem components

We introduce four sets representing the spaces of the variables of interest.

1. X is an input space. In MNISTAdd, this is the pair of MNIST digits $\mathbf{x} = (\mathcal{I}, \mathcal{J})$.
2. W is a structured space of k_W different discrete variables w_i , each with their own domain. Its elements $\mathbf{w} \in W = W_1 \times \dots \times W_{k_W}$ are *worlds* or *concepts* [59] of some $\mathbf{x} \in X$. For $(\mathcal{I}, \mathcal{J})$, the correct world is $\mathbf{w} = (5, 8) \in \{0, \dots, 9\}^2$.
3. Y is a structured space of k_Y discrete variables y_i . Elements $\mathbf{y} \in Y = Y_1 \times \dots \times Y_{k_Y}$ represent the output of the neurosymbolic system. Given world $\mathbf{w} = (5, 8)$, the sum is 13. We decompose the sum into individual digits, so $\mathbf{y} = (1, 3) \in \{0, 1\} \times \{0, \dots, 9\}$.
4. $\mathbf{P} \in \Delta^{|W_1|} \times \dots \times \Delta^{|W_{k_W}|}$, where each $\Delta^{|W_i|}$ is the probability simplex over the options of the variable w_i , is a *belief*. $\mathbf{P}$ assigns probabilities to different worlds with $p(\mathbf{w}|\mathbf{P}) = \prod_{i=1}^{k_W} \mathbf{P}_{i,w_i}$. That is, $\mathbf{P}$ is a parameter for an independent categorical distribution over the k_W choices.

We assume access to some *symbolic reasoning function* $c : W \rightarrow Y$ that deterministically computes the (structured) output $\mathbf{y}$ for any world $\mathbf{w}$. This function captures our background knowledge of the problem and we do not impose any constraints on its form. For MNISTAdd, c takes the two digits $(5, 8)$, sums them, and decomposes the sum by digit to form $(1, 3)$.

2.2 Weighted Model Counting

Together, these components form the *Weighted Model Counting* (WMC) problem [13]:

$$p(\mathbf{y}|\mathbf{P}) = \mathbb{E}_{p(\mathbf{w}|\mathbf{P})}[\mathbb{1}_{c(\mathbf{w})=\mathbf{y}}] \quad (1)$$

The WMC counts the *possible* worlds $\mathbf{w}^1$ for which c produces the output $\mathbf{y}$, and weights each possible world $\mathbf{w}$ with $p(\mathbf{w}|\mathbf{P})$. In PNL, we want to train a perception model f_θ to compute a belief $\mathbf{P} = f_\theta(\mathbf{x})$ for an input $\mathbf{x} \in X$ in which the correct world is likely. Note that *possible* worlds are not necessarily *correct* worlds: $\mathbf{w} = (4, 9)$ also sums to 13, but is not a symbolic representation of $\mathbf{x} = (\text{4}, \text{3})$.

Given this setup, we are interested in efficiently computing the following quantities:

1. $p(\mathbf{y}|\mathbf{P} = f_\theta(\mathbf{x}))$: We want to find the most likely outputs for the belief $\mathbf{P}$ that the perception network computes on the input $\mathbf{x}$.
2. $\nabla_{\mathbf{P}} p(\mathbf{y}|\mathbf{P} = f_\theta(\mathbf{x}))$: We want to train our neural network f_θ , which requires computing the gradient of the WMC problem.
3. $p(\mathbf{w}|\mathbf{y}, \mathbf{P} = f_\theta(\mathbf{x}))$: We want to find likely worlds given a predicted output and a belief about the perceived digits. The probabilistic logic programming literature calls $\mathbf{w}^* = \arg \max_{\mathbf{w}} p(\mathbf{w}|\mathbf{y}, \mathbf{P})$ the *most probable explanation (MPE)* [50].

2.3 The problem with exact inference for WMC

The three quantities introduced in Section 2.2 require calculating or estimating the WMC problem of Equation 1. However, the exact computation of the WMC is #P-hard, which is above NP-hard. Thus, we would need to do a potentially exponential-time computation for each training iteration and test query. Existing PNL methods use probabilistic circuits (PCs) to speed up this computation [60, 28, 35, 57]. PCs compile a logical formula into a circuit for which many inference queries are linear in the size of the circuit. PCs are a good choice when exact inference is required but do not overcome the inherent exponential complexity of the problem: the compilation step is potentially exponential in time and memory, and there are no guarantees the size of the circuit is not exponential.

The Multi-digit MNISTAdd task is excellent for studying the scaling properties of WMC. We can increase the complexity of MNISTAdd exponentially by considering not only the sum of two single digits (called $N = 1$) but the sum of two numbers with multiple digits. An example of $N = 2$ would be $\text{42} + \text{93} = 135$. There are 64 ways to sum to 135 using two numbers with two digits. Contrast this to summing two digits: there are only 6 ways to sum to 13. Each increase of N leads to 10 times more options to consider for exact inference. Our experiments in Section 4 will show that approximate inference can solve this problem up to $N = 15$. Solving this using exact inference would require enumerating around 10^{15} options for each query.

3 A-NESI: Approximate Neurosymbolic Inference

Our goal is to reduce the inference complexity of PNL to allow training neurosymbolic models on much more complex problems. To this end, in the following subsections, we introduce *Approximate Neurosymbolic Inference* (A-NESI). A-NESI approximates the three quantities of interest from Section 2, namely $p(\mathbf{y}|\mathbf{P})$, $\nabla_{\mathbf{P}} p(\mathbf{y}|\mathbf{P})$ and $p(\mathbf{w}|\mathbf{y}, \mathbf{P})$, using neural networks. We give an overview of our method in Figure 1.

3.1 Inference models

A-NESI uses an *inference model* $q_{\phi, \psi}$ defined as

$$q_{\phi, \psi}(\mathbf{w}, \mathbf{y}|\mathbf{P}) = q_{\phi}(\mathbf{y}|\mathbf{P})q_{\psi}(\mathbf{w}|\mathbf{y}, \mathbf{P}). \quad (2)$$

We call $q_{\phi}(\mathbf{y}|\mathbf{P})$ the *prediction model* and $q_{\psi}(\mathbf{w}|\mathbf{y}, \mathbf{P})$ the *explanation model*. The prediction model should approximate the WMC problem of Equation 1, while the explanation model should predict likely worlds $\mathbf{w}$ given outputs $\mathbf{y}$ and beliefs $\mathbf{P}$. One way to model $q_{\phi, \psi}$ is by considering W and Y as two sequences and defining an autoregressive generative model over these sequences:

$$q_{\phi}(\mathbf{y}|\mathbf{P}) = \prod_{i=1}^{k_Y} q_{\phi}(y_i|\mathbf{y}_{1:i-1}, \mathbf{P}), \quad q_{\psi}(\mathbf{w}|\mathbf{y}, \mathbf{P}) = \prod_{i=1}^{k_W} q_{\psi}(w_i|\mathbf{y}, \mathbf{w}_{1:i-1}, \mathbf{P}) \quad (3)$$

¹Possible worlds are also called ‘models’. We refrain from using this term to prevent confusion with ‘(neural network) models’.